Mini Project Report

on

ShopSphere - E-Commerce Platform

Under the Course

“ Database Management System Laboratory”

Submitted By

Vishal Ganbote	124B1F004
Atharva Bhargude	124B1F012
Shrejal Bhardwaj	124B1F019
Soham Chavan	124B1F022

Under the guidance of

Mrs. Alpana Borse

Department of Information Technology

Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Academic Year 2025-26

DATABASE MANAGEMENT SYSTEMS

PROJECT REPORT

ShopSphere

A Full-Stack E-Commerce Web Application

1. Executive Summary

ShopSphere is a production-ready, full-stack e-commerce web application designed to replicate the end-to-end shopping experience offered by leading online retail platforms. Built as a monorepo with a React frontend and a Node.js/Express backend, it leverages a relational MySQL database to manage all transactional and operational data. The system supports two primary roles — customer and administrator — each with a distinct, feature-rich interface.

From a Database Management Systems perspective, ShopSphere demonstrates the practical application of database design principles including normalization, referential integrity, indexing, and role-based data access. The database schema encompasses eight carefully designed tables that collectively handle user authentication, product cataloguing, inventory, cart management, order processing, wishlists, and customer reviews.

Key highlights of the project include:

- Normalized relational schema with eight core tables and optimized indexes
- JWT-based authentication with bcrypt password hashing and role separation
- Real-time capabilities via Socket.io for live order and notification updates
- RESTful API with seven module-level route groups served by Express.js
- Responsive, mobile-first React 18 UI powered by Vite
- Full admin dashboard with analytics, product CRUD, and order oversight

2. Project Objectives

The primary objectives of the ShopSphere project are:

- **Design and implement a normalized relational database schema suitable for an e-commerce domain**
  - Ensure data integrity through primary and foreign key constraints
  - Apply appropriate indexing strategies to optimize query performance
- **Develop a secure, role-aware backend API that reflects real-world database interaction patterns**
  - Implement authentication and authorization using JWT tokens
  - Enforce input validation and sanitization to prevent SQL injection and data corruption
- **Build a complete, user-centric frontend application that communicates with the backend API**
  - Support responsive design for mobile, tablet, and desktop viewports
  - Provide real-time feedback through WebSocket integration
- **Demonstrate full CRUD operations across all major entities (users, products, orders, reviews)**
- **Apply software engineering best practices: modular architecture, separation of concerns, environment-based configuration**

3. Technology Stack

ShopSphere employs a modern, industry-standard technology stack across its three tiers: presentation, business logic, and data.

Layer	Technology	Purpose / Notes
Frontend Framework	React 18	Component-based UI with hooks and context API
Routing	React Router	Client-side declarative routing
Build Tool	Vite	Fast HMR in development; optimized production builds
Styling	Custom CSS3	Animations, transitions, responsive grid layouts
Runtime	Node.js (ES6+)	Server-side JavaScript with module support
API Framework	Express.js	RESTful API definition, middleware pipeline
Database	MySQL 8.0	Relational DBMS; central to all data operations
DB Driver	mysql2	Promise-based MySQL connector for Node.js
Authentication	JWT + bcrypt	Stateless tokens; password hashing with salt rounds
Real-time	Socket.io	WebSocket-based live notifications and updates
Security	Helmet + CORS	HTTP security headers; cross-origin policy control
Validation	express-validator	Server-side schema-level input validation
Dev Tools	Nodemon + Morgan	Auto-restart on change; HTTP request logging

4. Database Design

4.1 Schema Overview

The ShopSphere MySQL database is designed following the principles of the Third Normal Form (3NF) to eliminate redundancy and ensure data consistency. All inter-entity relationships are enforced through foreign key constraints, and composite lookups are accelerated by strategic secondary indexes.

4.2 Entity Descriptions

The schema comprises eight core tables, each mapped to a distinct domain entity:

Table	Description
users	Stores customer and admin account data including hashed passwords, role flags, and profile information. Role field distinguishes between 'customer' and 'admin' access levels.
categories	Maintains product classification with human-readable names and unique URL slugs for SEO-friendly routing.
products	Central product catalog recording name, description, price, stock quantity, average ratings, and foreign key to categories.
orders	Captures order-level metadata: associated user, total amount, payment status, shipping status, and delivery address.
order_items	Junction table linking orders to specific products with quantity and unit price at the time of purchase.
cart_items	Tracks per-user shopping cart state, associating users with products and their selected quantities.
wishlist	Records user product bookmarks (wishlist entries) as a many-to-many relationship between users and products.
reviews	Stores product reviews submitted by customers including a numeric rating and textual content.

4.3 Relationships and Referential Integrity

The following entity relationships define the relational structure:

- users → orders: One-to-many (a user can place multiple orders)
- orders → order_items: One-to-many (an order contains one or more items)
- products → order_items: One-to-many (a product can appear in many order items)
- users → cart_items: One-to-many (a user can have multiple items in their cart)
- products → cart_items: One-to-many (a product can be in multiple users' carts)
- users → wishlist: One-to-many (a user can wishlist multiple products)
- products → wishlist: One-to-many (a product can be wishlisted by multiple users)

- users → reviews: One-to-many (a user can write multiple reviews)
- products → reviews: One-to-many (a product can receive multiple reviews)
- categories → products: One-to-many (a category groups multiple products)

4.4 Indexing Strategy

To optimize query performance, secondary indexes are created on frequently filtered and joined columns. The following fields are indexed:

- products.name — supports full-text and LIKE-based search queries
- products.category_id — accelerates category-filter joins
- orders.user_id — speeds up user order history retrieval
- reviews.product_id — improves product review aggregation

These indexes complement the primary key indexes and significantly reduce table scan overhead for the most common API query patterns.

5. System Architecture

5.1 High-Level Architecture

ShopSphere follows a three-tier architecture: a React-based presentation tier, a Node.js/Express application tier, and a MySQL data tier. The frontend and backend are developed and deployed independently, communicating exclusively through a well-defined RESTful API. Socket.io provides an additional real-time channel over WebSockets.

5.2 Project Directory Structure

Path	Contents
client/src/components/	Reusable UI components (Navbar, ProductCard, Modal, etc.)
client/src/pages/	Top-level page components mapped to routes
client/src/context/	React Context providers for global state (auth, cart)
client/src/hooks/	Custom hooks abstracting data-fetching and logic
client/src/api/	Axios-based API client functions per domain
client/src/styles/	Modular CSS stylesheets with animations
server/src/controllers/	Route handler functions (business logic entry points)
server/src/models/	Database query functions (direct MySQL interaction)
server/src/routes/	Express router definitions grouping related endpoints
server/src/middleware/	Auth guards, error handlers, validation middleware
server/src/services/	Reusable business logic services (e.g., email, payment)
database/schema.sql	DDL for all 8 tables, constraints, and indexes
database/seed.sql	Sample data for development and testing

5.3 API Route Groups

The Express backend exposes seven primary route namespaces, each delegating to a dedicated controller module:

Route Prefix	Responsibility
/auth	User registration, login, token refresh, and password reset
/products	Product listing, search, filtering by category, and detail retrieval
/users	Profile management, address book, and account settings

Route Prefix	Responsibility
/cart	Add, update quantity, remove items, and fetch cart contents
/wishlist	Add/remove products to wishlist, retrieve user wishlist
/orders	Place orders, retrieve order history, and check order status
/reviews	Submit and retrieve product reviews and ratings
/admin	Analytics dashboard data, admin product CRUD, user oversight

6. Features and Functionality

6.1 Customer-Facing Features

Authentication & Profile

- User registration with email, password (bcrypt hashed), and profile details
- Login with JWT token issuance; token stored client-side for subsequent requests
- Profile management: update name, address, and password

Product Discovery

- Homepage with product carousels and promotional banners
- Category-based browsing with dynamic filtering
- Live search with instant suggestions as the user types
- Detailed product pages showing images, description, price, stock status, and aggregated ratings

Shopping Experience

- Add-to-cart with per-item quantity controls
- Wishlist for saving products across sessions
- Checkout flow with address confirmation and mock payment integration
- Order history with status tracking per order

Reviews & Ratings

- Authenticated users can submit a rating (1–5 stars) and written review
- Aggregated average rating stored on the product record for fast reads

6.2 Administrator Features

- Secure admin role gated behind role-based JWT middleware
- Analytics dashboard: total revenue, order counts, top-selling products
- Product management: create, edit, and delete products; update stock levels
- Order management: view all orders and update fulfillment/payment status
- User oversight: view registered users and manage account status

6.3 Real-Time Features

Socket.io is integrated on both the server and client to push live event notifications. When an order status changes or a new order is placed, connected clients receive an immediate update without polling. This demonstrates the integration of WebSocket communication alongside a traditional REST API.

7. Security Implementation

Security is treated as a first-class concern across both the database and application layers. The following measures are implemented:

Security Measure	Implementation Detail
Password Hashing	bcrypt with configurable salt rounds; plaintext passwords are never stored
Stateless Auth (JWT)	Tokens carry user identity and role; validated on every protected request via middleware
Role-Based Access Control	Admin routes protected by role guard middleware; customer tokens cannot access admin endpoints
Input Validation	express-validator enforces schema rules on all request bodies; malformed inputs are rejected before reaching DB
SQL Injection Prevention	Parameterized queries via mysql2 prepared statements; no raw string interpolation in SQL
HTTP Security Headers	Helmet middleware sets Content-Security-Policy, X-Frame-Options, and other protective headers
CORS Policy	Cross-Origin Resource Sharing restricted to trusted origins; prevents unauthorized API access from third-party domains
Token Expiry	JWT tokens have configurable expiration; expired tokens are rejected, requiring re-authentication

8. Setup and Deployment

8.1 Prerequisites

- Node.js v16 or higher
- MySQL 8.0 server (local or cloud-hosted)
- npm or yarn package manager

8.2 Installation Procedure

Step	Action	Detail
1	Clone Repository	git clone <repository-url> and navigate into the project root
2	Database Setup	Create a MySQL database; execute schema.sql to create tables and seed.sql to populate test data
3	Environment Config	Create .env files in the server directory with DB host, port, name, credentials, and JWT secret
4	Install Dependencies	Run npm install in both client/ and server/ directories (or use root-level concurrent script)
5	DB Initialization	Run npm run setup-db in server/ to verify schema and apply any automated initialization
6	Start Application	Launch frontend on port 5173 (Vite dev server) and backend on port 5000 (Express)

8.3 Default Test Accounts

The seed data includes two pre-configured accounts for testing:

Role	Email	Password
Admin	admin@shopsphere.com	password123
Customer	ava@example.com	password123

Note: Test account credentials should be changed before any production deployment.

9. Performance and Scalability

ShopSphere is designed with performance as a key architectural concern, particularly at the database layer where query efficiency directly impacts user experience:

- **Database Indexing: Secondary indexes on product name, category ID, user-order associations, and review foreign keys reduce full table scans for the most common query paths.**
- **Stateless Backend: JWT-based authentication eliminates server-side session storage, allowing the API layer to scale horizontally without shared session state.**
- **Optimized Frontend Builds: Vite produces tree-shaken, minified production bundles with code-splitting, reducing initial load times.**
- **Modular Architecture: Controllers, models, services, and routes are fully decoupled, enabling independent scaling or replacement of individual components.**
- **WebSocket Efficiency: Real-time updates are pushed via Socket.io rather than polling, reducing unnecessary HTTP overhead for live data.**

10. DBMS Concepts Demonstrated

ShopSphere serves as a practical demonstration of the following core DBMS concepts covered in the curriculum:

DBMS Concept	Demonstration in ShopSphere
Data Definition Language (DDL)	schema.sql uses CREATE TABLE, PRIMARY KEY, FOREIGN KEY, and CREATE INDEX statements
Data Manipulation Language (DML)	API models use SELECT, INSERT, UPDATE, DELETE via parameterized queries
Normalization (up to 3NF)	Entities decomposed to eliminate transitive dependencies; e.g., order_items separates item price from product price
Referential Integrity	FOREIGN KEY constraints with ON DELETE CASCADE / RESTRICT rules across all relationships
Indexing	B-Tree secondary indexes on high-cardinality filter and join columns
Transactions	Order placement wraps insert into orders and order_items in a single atomic transaction
Role-Based Access	Database-level and application-level separation of customer vs. admin data access
Aggregation	Average product rating computed via SQL AVG() and stored on product record; order totals via SUM()
Joins	Multi-table SELECT queries join users, orders, order_items, and products for order history views
Prepared Statements	All user-supplied values bound via mysql2 parameterized queries, preventing SQL injection

11. Conclusion

ShopSphere successfully demonstrates the design and implementation of a production-grade relational database at the core of a full-stack web application. The project integrates eight normalized MySQL tables, enforced through referential integrity constraints and enhanced by targeted indexing, to support a comprehensive e-commerce domain.

Beyond the database layer, the project showcases how modern application development practices — JWT authentication, RESTful API design, real-time WebSockets, and modular architecture — complement a well-designed data model to produce a scalable, secure, and maintainable system.

The application provides a concrete, end-to-end illustration of DBMS concepts including normalization, joins, transactions, aggregation, prepared statements, and role-based access control — making it both an academically rigorous and practically relevant project.

Future enhancements could include migration to a connection-pooling ORM, full-text search indexing, caching layers via Redis, and a CI/CD pipeline with automated schema migrations — all of which would further reinforce database engineering best practices.